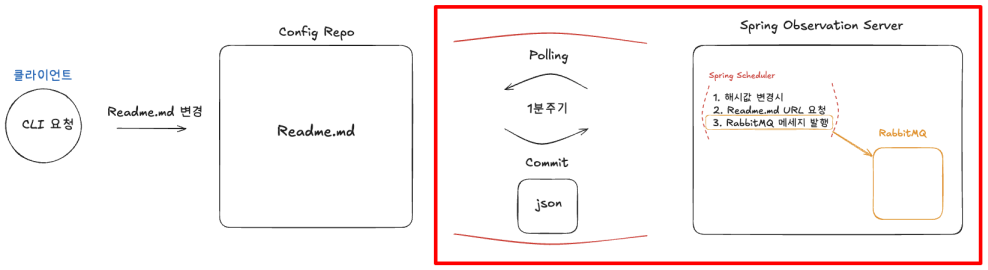
2. 관찰서버 만들기

1. 깃헙 레포지토리 생성하기

깃헙에서 config-readme 레포를 생성 후

1분마다 GitHub의 README.md를 감시해서,

변경이 발생하면 **RabbitMQ**로 메시지를 보내는 서버를 만듭니다.



---

1. Sprign 프로젝트 생성

항목	설정
Project	Gradle - Groovy
Language	Java
Spring Boot	3.3.x 이상
Group	com.rabbitmq
Artifact	observation
Java Version	21
Packaging	Jar
Dependencies	Spring Web Spring for RabbitMQ Lombok

---

2. 설정 파일(application.properties)

```
server.port=8080

github.owner=kjh5848
github.repo=config-readme
github.readme-path=README.md
github.poll-interval-ms=60000

spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
```

```
spring.rabbitmq.password=guest

rabbit.exchange=github.events
rabbit.queue=repo-updates
rabbit.routing-key=readme.changed
```

💡 github.owner=
github.repo=
github.readme-path=
실습시 해당 정보를 본인 깃헙 레포지토리 정보와 동일하게 작성하여야합니다.

3.패키지 구조

```
com.rabbitme.observer
 ├ config/RabbitConfig.java
 ├ github/GitHubClient.java
 ├ scheduler/PollingScheduler.java
 ├ rabbit/RabbitProducer.java
 └ dto/RabbitDTO.java
```

---

1) RabbitMQ 설정하기 (RabbitConfig.java)

💡 RabbitMQ는 **Exchange ↔ Queue ↔ RoutingKey** 개념으로 메시지를 전달합니다.
따라서 스프링이 자동으로 연결할 수 있도록 구성해야 합니다.

항목	설명
Exchange	메시지를 받아 라우팅하는 허브
Queue	실제 메시지를 저장하는 통
RoutingKey	어떤 큐로 보낼지 구분하는 키

코드

경로: config/RabbitConfig.java

```
package com.rabbitmq.observer.config;

import org.springframework.amqp.core.*;
import org.springframework.amqp.support.converter.Jackson2JsonMessageConverter;
import org.springframework.amqp.support.converter.MessageConverter;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

// RabbitMQ 교환기, 큐, 라우팅키를 스프링 Bean으로 등록
@Configuration
```

```
public class RabbitConfig {

    @Value("${rabbit.exchange}")
    private String exchangeName;

    @Value("${rabbit.queue}")
    private String queueName;

    @Value("${rabbit.routing-key}")
    private String routingKey;

     // Jackson2 기반 JSON 컨버터 등록
    @Bean
    public MessageConverter jsonMessageConverter() {
        return new Jackson2JsonMessageConverter();
    }

    @Bean
    public TopicExchange exchange() {
        return new TopicExchange(exchangeName, true, false);
    }

    @Bean
    public Queue queue() {
        return QueueBuilder.durable(queueName).build();
    }

    @Bean
    public Binding binding(Queue queue, TopicExchange exchange) {
        return BindingBuilder.bind(queue).to(exchange).with(routingKey);
    }
}
```

---

2) 메시지 데이터 구조 정의 (ReadmeDTO.java)

💡 메시지를 보낼 때 일정한 구조를 가져야 JSON으로 변환할 수 있습니다.

필드	의미	
repo	감시 중인 GitHub 저장소 이름	
sha	변경된 커밋 SHA	
content	변경된 README 내용	
timestamp	변경 시각	

코드

경로: dto/RabbitDTO.java

```
package com.rabbitmq.observer.dto;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class RabbitDTO {
    private String repo;
    private String sha;
    private String content;
    private LocalDateTime timestamp;
}
```

---

3) RabbitMQ 발행 클래스 (RabbitProducer.java)

> 💡 이 클래스는 메시지를 **JSON으로 변환 → RabbitMQ에 발행**하는 역할을 합니다.
> Spring Boot는 Jackson을 이용해 자동으로 직렬화를 처리합니다.

코드

경로: rabbit/RabbitProducer.java

```
package com.rabbitmq.observer.rabbit;

import com.rabbitmq.observer.dto.RabbitDTO;
import lombok.RequiredArgsConstructor;

import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

@Component
@RequiredArgsConstructor
public class RabbitProducer {

    private final RabbitTemplate rabbitTemplate;

    @Value("${rabbit.exchange}")
    private String exchange;

    @Value("${rabbit.routing-key}")
```

```
    private String routingKey;

    public void send(RabbitDTO message) {
        rabbitTemplate.convertAndSend(exchange, routingKey, message);
        System.out.println("[RabbitMQ] 메시지 발행 → " + message);
    }
}
```

---

4) GitHub API 호출 (GitHubClient.java)

💡 Observation Server는 GitHub의 README 상태를 알아야 합니다.
이를 위해 GitHub REST API를 호출합니다.

목적	URL 예시
최신 SHA 조회	https://api.github.com/repos/{owner}/{repo}/commits?path=README.md&per_page=1
README 내용	https://raw.githubusercontent.com/{owner}/{repo}/main/README.md

---

코드

경로: github/GitHubClient.java

```
package com.rabbitmq.observer.github;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;
import org.springframework.web.client.RestTemplate;
import org.springframework.http.*;
import java.util.*;

@Component
public class GitHubClient {

    @Value("${github.owner}")
    private String owner;

    @Value("${github.repo}")
    private String repo;

    @Value("${github.readme-path}")
    private String readmePath;

    private final RestTemplate restTemplate = new RestTemplate();

    // 최신 커밋 SHA 가져오기
    public String fetchLatestSha() {
        String url = String.format(
                "https://api.github.com/repos/%s/%s/commits?path=%s&per_page=1",
                owner, repo, readmePath
        );
```

```
        HttpHeaders headers = new HttpHeaders();

        ResponseEntity<List> response =
                restTemplate.exchange(url, HttpMethod.GET, new HttpEntity<>(headers), List.class);

        if (response.getBody() != null && !response.getBody().isEmpty()) {
            Map<?, ?> commitObj = (Map<?, ?>) response.getBody().get(0);
            return (String) commitObj.get("sha");
        }
        return null;
    }

    // README 내용 가져오기
    public String fetchReadmeContent() {
        String rawUrl = String.format(
                "https://raw.githubusercontent.com/%s/%s/main/%s",
                owner, repo, readmePath
        );
        return restTemplate.getForObject(rawUrl, String.class);
    }
}
```

5) 폴링 로직 구현 (ObserverApplication.java/PollingScheduler.java)

💡 스프링 부트 애플리케이션은 항상 진입점(main())이 필요합니다.
또한, 이 서버는 **정기 실행 작업(Scheduler)** 을 포함하므로
@EnableScheduling 어노테이션을 활성화해야 합니다.

코드

경로: ObserverApplication.java

```
package com.rabbitmq.observer;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableScheduling;

@SpringBootApplication
@EnableScheduling
public class ObserverApplication {

    public static void main(String[] args) {
        SpringApplication.run(ObserverApplication.class, args);
    }

}
```

💡 1분마다 GitHub의 최신 SHA를 비교해서
변경되면 RabbitMQ로 메시지를 보내야 합니다.
lastSha에 메모리로 저장 후 이후 변경된 커밋의 SHA와 비교합니다.

코드

경로: scheduler/PollingScheduler.java

```
package com.rabbitmq.observer.scheduler;

import com.rabbitmq.observer.dto.RabbitDTO;
import com.rabbitmq.observer.github.GitHubClient;
import com.rabbitmq.observer.rabbit.RabbitProducer;

import lombok.RequiredArgsConstructor;

import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;
import java.time.LocalDateTime;

@Component
@RequiredArgsConstructor
public class PollingScheduler {

    private final GitHubClient gitHubClient;
    private final RabbitProducer rabbitProducer;

    // ✅ 마지막으로 확인한 커밋 해시 저장
    private String lastSha = null;

    // ✅ 1분마다 실행 (github.poll-interval-ms 설정 기반)
    @Scheduled(fixedRateString = "${github.poll-interval-ms}")
    public void checkForReadmeChange() {
        try {
            // ① 최신 README 커밋 SHA 가져오기
            String latestSha = gitHubClient.fetchLatestSha();
            if (latestSha == null) {
                System.out.println("[폴링] 커밋 SHA를 가져오지 못했습니다.");
                return;
            }

            // ② 이전 SHA와 비교 → 변경 없으면 종료
            if (latestSha.equals(lastSha)) {
                System.out.println("[폴링] 변경 없음 → SHA: " + latestSha);
                return;
            }

            // ③ 변경 감지 시 SHA 갱신
            lastSha = latestSha;
            System.out.println("[폴링] README 변경 감지 → 새로운 SHA: " + latestSha);
```

```
        // ④ README 파일 내용(raw.githubusercontent.com) 가져오기
        String content = gitHubClient.fetchReadmeContent();

        // ⑤ RabbitMQ로 발행할 DTO 생성
        RabbitDTO message = RabbitDTO.builder()
            .repo(gitHubClient.getRepoFullName())
            .sha(latestSha)
            .content(content)
            .timestamp(LocalDateTime.now())
            .build();

        // ⑥ RabbitMQ에 발행
        rabbitProducer.send(message);

    } catch (Exception e) {
        System.err.println("[ERROR] 폴링 중 오류: " + e.getMessage());
    }
  }
}
```

4 실행 및 결과 확인

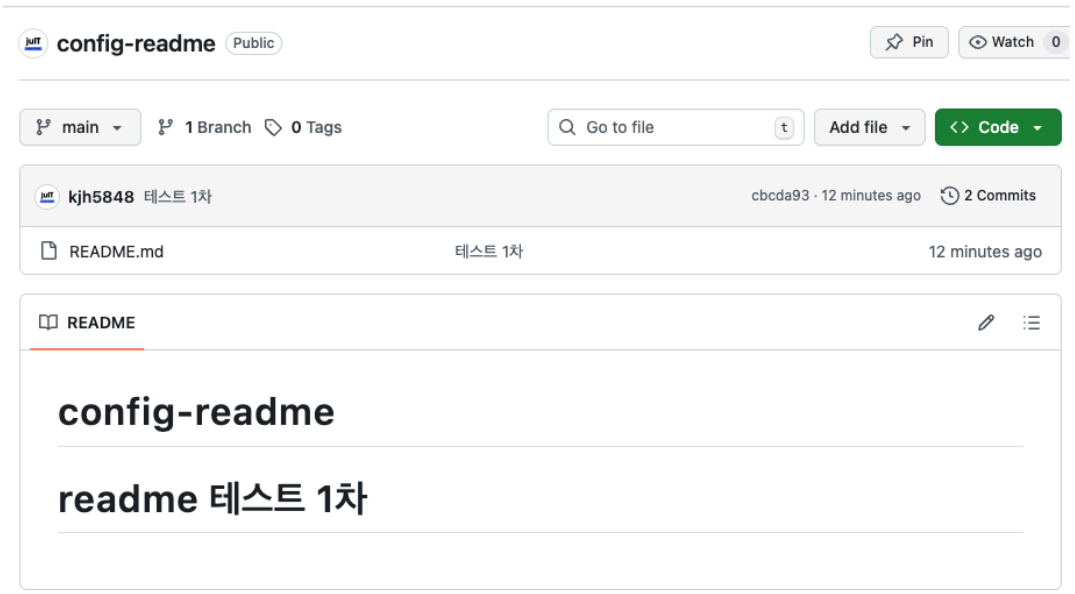
1) RabbitMQ 실행

```
docker run -d --name rabbit -p 5672:5672 -p 15672:15672 rabbitmq:management
```

2) 서버 실행

```
./gradlew bootRun
```

3) 로그 확인



readme 테스트 1차 까지 있던 커밋 내용을 로그에서 확인할 수 있으며, 해당 SHA를 메모리에 저장 후 앞으로 변경된 커밋 SHA와 비교하여 메세지를 발행합니다.

```
text path ")"
2025-10-10T16:52:11.328+09:00  INFO 15320 --- [observer] [           main] c.rabbitmq.observer.ObserverApplication  : Started ObserverApplication in 1.167 second
s (process running for 1.323)
[폴링] README 변경 감지 → 새로운 SHA: cbcda934db14c5ea51a56c52e65c61068bb2258c
2025-10-10T16:52:12.250+09:00  INFO 15320 --- [observer] [   scheduling-1] o.s.a.r.c.CachingConnectionFactory       : Attempting to connect to: [localhost:5672]
2025-10-10T16:52:12.277+09:00  INFO 15320 --- [observer] [   scheduling-1] o.s.a.r.c.CachingConnectionFactory       : Created new connection: rabbitConnectionFac
tory#1f1e58ca:0/SimpleConnection@3f965141 [delegate=amqp://guest@127.0.0.1:5672/, localPort=51954]
[RabbitMQ] 메시지 발행 → RabbitDTO(repo=kjh5848/config-readme, sha=cbcda934db14c5ea51a56c52e65c61068bb2258c, content=# config-readme
# readme 테스트 1차
, timestamp=2025-10-10T16:52:12.235962)
[폴링] 변경 없음 → SHA: cbcda934db14c5ea51a56c52e65c61068bb2258c
```

> [폴링] README 변경 감지 → 새로운 SHA: cbcda934db14c5ea51a56c52e65c61068bb2258c
> [RabbitMQ] 메시지 발행 → RabbitDTO(repo=kjh5848/config-readme, sha=cbcda934db14c5ea51a56c52e65c61068bb2258c, content=# config-readme
> # readme 테스트 1차
> , timestamp=2025-10-10T16:52:12.235962)
> [폴링] 변경 없음 → SHA: cbcda934db14c5ea51a56c52e65c61068bb2258c

4) RabbitMQ 콘솔 확인

레빗 콘솔에서도 저장된것을 확인할 수 있습니다.

<u>3. 구독서버 만들기</u>